Università di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica



ChatCart: agente conversazionale con interfaccia generativa

Tesi di Laurea

Laureando

Edoardo De Piccoli

Matricola 2101055

Relatore

Prof. Tullio Vardanega

Anno Accademico 2025-2026

© Edoardo De Piccoli. Tutti i diritti riservati. Tesi di Laurea: “ChatCart: agente conversazionale con interfaccia generativa”, Università di Padova, Dipartimento di Matematica “Tullio Levi-Civita”, Corso di Laurea in Informatica.

Sommario

In questa relazione descrivo lo *stage* che ho svolto presso Halue S.r.l., società di consulenza tecnologica con sede operativa ad Abano Terme, dal 14 settembre al 9 novembre 2026.

Con la supervisione del Prof. Tullio Vardanega, relatore, e di Antonio Cavestro, *tutor* aziendale, ho progettato e realizzato ChatCart, il prototipo di un assistente conversazionale per i negozi Shopify che risponde al cliente non soltanto con testo, ma anche con componenti dell'interfaccia che sceglie e popola in tempo reale con i dati del negozio.

Struttura del documento. La relazione si articola in quattro capitoli:

- **Capitolo 1: «Halue e il commercio digitale».** Presento l'azienda, la sua clientela, le tecnologie e i processi che adotta e il suo rapporto con l'innovazione, in particolare con l'intelligenza artificiale.
- **Capitolo 2: «Oltre la conversazione testuale».** Discuto il problema che lo *stage* affronta, il suo posto nella strategia di Halue, gli obiettivi e i vincoli del progetto e le ragioni per cui l'ho scelto.
- **Capitolo 3: «Progetto e sviluppo di ChatCart».** Illustro il metodo di lavoro, l'analisi, l'architettura, la realizzazione dell'agente e del *widget*, le funzionalità sperimentali e le possibili evoluzioni del progetto, la verifica e i risultati ottenuti.
- **Capitolo 4: «Valutazione retrospettiva».** Valuto il grado di soddisfacimento degli obiettivi, la mia maturazione professionale e la distanza tra le competenze che lo *stage* mi ha richiesto e quelle che ho acquisito nel corso di studi.

In appendice raccolgo gli acronimi e le abbreviazioni con la loro forma estesa, il glossario e i riferimenti.

Convenzioni tipografiche. Nel documento adotto le seguenti convenzioni:

- il corsivo indica i termini in lingua straniera, e soltanto questi; i nomi propri di aziende, prodotti e metodi (come Shopify, Slack o Scrum) restano in tondo;
- alla prima occorrenza di un termine presente nel glossario ne do nel testo una definizione essenziale e lo contrassegno con il pedice G; il termine e il pedice sono cliccabili e rimandano all'approfondimento nel glossario;
- le figure e le tabelle hanno una numerazione progressiva e una didascalia, che ne indica la fonte quando non sono di mia realizzazione.

Indice

1 Halue e il commercio digitale **1**

1.1 Profilo aziendale e linee di attività 1

1.2 Clientela e modalità di ingaggio 3

1.3 Ecosistema tecnologico 5

1.4 Processi di sviluppo e gestione dei progetti 6

1.4.1 Organizzazione del lavoro e gestione dei progetti 6

1.4.2 Dal *ticket* al rilascio 9

1.5 Innovazione e intelligenza artificiale 11

2 Oltre la conversazione testuale **13**

2.1 Il ruolo degli *stage* nella strategia aziendale 13

2.2 I limiti degli assistenti conversazionali testuali 13

2.3 Il commercio agentico nella visione di Halue 14

2.4 Obiettivi e vincoli del progetto 14

2.5 Motivazioni e obiettivi personali 14

3 Progetto e sviluppo di ChatCart **15**

3.1 Metodo di lavoro 15

3.1.1 Pianificazione e interazione con il *tutor* aziendale 15

3.1.2 Organizzazione del lavoro e tracciamento delle attività 15

3.1.3 Strumenti e assistenti di programmazione 16

3.2 Studio preliminare e analisi 16

3.2.1 L'ecosistema delle applicazioni Shopify 16

3.2.2 Un'analisi dei requisiti iterativa 16

3.3 Architettura del sistema 16

3.3.1 Visione d'insieme 16

3.3.2 Il contratto tra agente e *widget* 17

3.4 Realizzazione 17

3.4.1 L'agente conversazionale 17
3.4.2 Il *widget* e l'integrazione con il tema del negozio 17
3.4.3 Metriche di utilizzo e analisi delle conversazioni 17
3.5 Sperimentazioni e sviluppi futuri 18
3.6 Verifica e validazione 18
3.7 Risultati 18
3.7.1 Il prodotto dal punto di vista dell'utente 18
3.7.2 Risultati quantitativi 18

4 Valutazione retrospettiva **20**
4.1 Soddisfacimento degli obiettivi 20
4.2 Maturazione professionale 20
4.3 Formazione universitaria e lavoro 21

Acronimi e abbreviazioni **22**

Glossario **23**

Riferimenti **28**

Elenco delle figure

1.1 Le linee di attività di Halue, in ordine di peso dall'*e-commerce* alla progettazione dell'interfaccia. La consulenza tecnologica e l'intelligenza artificiale le attraversano tutte. 3

1.2 I momenti di coordinamento del lavoro in Halue, ordinati per frequenza: la riunione quotidiana, le attività di ogni *sprint*, la pianificazione mensile delle persone sui clienti e i rapporti di avanzamento e le retrospettive di ogni progetto. 7

1.3 Il ciclo di vita di un *ticket* in Halue: dalla richiesta del cliente al *ticket* su Jira, attraverso i due *refinement*, la stima, lo sviluppo con *pull request* e *test*, il collaudo in *staging*, fino al rilascio in produzione e alla manutenzione. 10

Elenco delle tabelle

1.1 I progetti per i clienti che Halue pubblica sul proprio sito, con il settore del cliente, la piattaforma e il tipo di intervento. 4
1.2 Le tecnologie ricorrenti nei progetti di Halue e il loro utilizzo nel progetto di *stage*. 5
1.3 Gli strumenti di lavoro di Halue, con il loro uso e il modo in cui li conosco: usati durante lo *stage*, visti usare dai colleghi o soltanto riferiti dal *tutor*. 8

Elenco dei codici sorgente

Capitolo 1

Halue e il commercio digitale

Presento Halue S.r.l., l'azienda presso cui ho svolto lo *stage*, descrivendone le attività, i clienti, le tecnologie, i processi e il rapporto con l'innovazione. Mi baso sulle otto settimane di lavoro, sulle conversazioni con i colleghi e sulle risposte raccolte durante un'intervista dedicata al *tutor* aziendale, Antonio Cavestro, integrando queste informazioni con quelle del sito dell'azienda[1]. Le risposte hanno confermato quanto ho osservato e completato il quadro sugli aspetti che non ho seguito direttamente, come contratti e gestione dei progetti; per questi aspetti riporto informazioni riferite, mentre distinguo le dichiarazioni del sito dalle mie osservazioni.

1.1 Profilo aziendale e linee di attività

Halue è una società di consulenza tecnologica nata nel 2022, con sede legale a Parma e sede operativa ad Abano Terme, in provincia di Padova. Dal 2024 è una Società Benefit$_G$: una forma giuridica, introdotta in Italia nel 2015, per le società che oltre al profitto perseguono obiettivi di beneficio comune dichiarati nello statuto e ne rendono conto ogni anno in una relazione d'impatto.

Oggi l'azienda conta dieci persone, con i seguenti ruoli:

- *Founder & Senior Consultant*$_G$: il fondatore Valentino Baraldo, che svolge anche attività di consulenza;
- *Solution & Technical Architect*$_G$: definisce le soluzioni e l'architettura tecnica;

[1] Halue S.r.l., sito istituzionale, https://halue.com/it, pagine «Chi siamo», «Approccio» e «Progetti», consultate il 4 novembre 2026.

- *Senior Project Manager*$_G$ e *Project Management Assistant*$_G$: due ruoli dedicati alla gestione dei progetti;
- *Senior Software Engineer*$_G$, *Software Engineer*$_G$ e *Junior Developer*$_G$: cinque sviluppatori con diversi livelli di esperienza;
- *UX/UI Digital Consultant*$_G$: cura la progettazione dell'esperienza utente e dell'interfaccia.

Halue è nata come società di consulenza sull'*e-commerce*$_G$ e sui sistemi di *Customer Relationship Management* (CRM)$_G$, con Shopify e Salesforce come piattaforme di riferimento. Il primo ambito riguarda la vendita di prodotti e servizi tramite canali digitali; il secondo, la gestione delle relazioni di un'azienda con i propri clienti. Halue dedica inoltre particolare attenzione allo sviluppo di soluzioni basate sull'intelligenza artificiale per il commercio digitale. In questo ambito, un modello linguistico di grandi dimensioni (LLM)$_G$, addestrato a elaborare e generare testo, può assistere il cliente nella consultazione di cataloghi complessi e nel percorso d'acquisto attraverso l'accesso ai dati del negozio.

La figura 1.1 riassume le linee di attività dell'azienda, in ordine di peso:

1. l'*e-commerce* su Shopify e su Salesforce Commerce Cloud, sia verso il consumatore finale (B2C$_G$) sia verso altre aziende (B2B$_G$);
2. l'integrazione di sistemi, cioè il collegamento del negozio con i gestionali aziendali (ERP$_G$), la logistica e i servizi di terze parti, punto a punto o tramite piattaforme di integrazione (iPaaS$_G$);
3. i servizi di intelligenza artificiale, dalla valutazione di quanto un'azienda sia pronta ad adottarla fino allo sviluppo di agenti su misura;
4. i sistemi CRM su Salesforce;
5. la formazione;
6. la progettazione dell'interfaccia e dell'esperienza utente.

La figura mostra due elementi trasversali. La consulenza tecnologica accompagna tutte le linee, dalla valutazione iniziale alla scelta dei sistemi. L'intelligenza artificiale è insieme una linea a sé e, sempre di più, una competenza che entra nei progetti di *e-commerce* e di integrazione.

Halue lavora su commessa, non vende prodotti propri, come applicazioni pubblicate sullo Shopify App Store, ma costruisce soluzioni su misura per i singoli clienti. Anche ChatCart segue questa impostazione: descrivo nella sezione 3.2.1 come lo integro nel singolo negozio, anziché distribuirlo come applicazione pubblica.



Figura 1.1: Le linee di attività di Halue, in ordine di peso dall'*e-commerce* alla progettazione dell'interfaccia. La consulenza tecnologica e l'intelligenza artificiale le attraversano tutte.

1.2 Clientela e modalità di ingaggio

Halue lavora soltanto con aziende private, per lo più di medie e grandi dimensioni, e non con la pubblica amministrazione. Sono marchi strutturati della moda, dello sport, del commercio al dettaglio e della cosmetica professionale, che vendono *online* su più mercati, con lingue e valute diverse. La maggior parte dei clienti è italiana, ma ci sono anche marchi esteri.

Il sito di Halue cita alcuni di questi clienti. La tabella 1.1 riporta i quattro progetti che descrive; oltre a questi, il sito mostra i marchi di Piazza Italia, Velasca, Davines e Comfort Zone.

Tabella 1.1: I progetti per i clienti che Halue pubblica sul proprio sito, con il settore del cliente, la piattaforma e il tipo di intervento.

Cliente	**Settore**	**Piattaforma**	**Intervento**
Dainese	Abbigliamento e dispositivi di protezione per il motociclismo	Salesforce Commerce Cloud	Revisione dell'architettura per la gestione di più marchi
Philippe Model Paris	Calzature e abbigliamento	Shopify Plus	Migrazione della piattaforma di commercio elettronico da Adobe Commerce
Colnago	Produzione di biciclette	Shopify Plus	Sviluppo di un configuratore tridimensionale per la personalizzazione dei prodotti
Gruppo Rossignol	Attrezzature per sport invernali	Salesforce Commerce Cloud	Realizzazione dei negozi digitali dei marchi Dynastar e Lange

La tabella mostra che la clientela si divide tra le due piattaforme principali, Shopify e Salesforce, e che gli interventi vanno dalla migrazione di un intero negozio a funzionalità molto specifiche, come un configuratore tridimensionale.

I progetti arrivano soprattutto per passaparola, da clienti e contatti che segnalano l'azienda, e in parte da gare e bandi. Le formule contrattuali sono tre: a tempo e materiali, in cui il cliente paga il tempo effettivamente impiegato; a corpo, con un prezzo fissato in anticipo, per i progetti dal perimetro ben definito; e i contratti di manutenzione e assistenza per i negozi già in produzione. Halue è inoltre *partner* di Shopify per Shopify Plus, il piano della piattaforma rivolto ai commercianti più grandi, sia per i negozi B2C sia per quelli B2B.

Due caratteristiche di questa clientela ritornano nel resto della relazione. La prima è la vendita su più mercati: per questo il supporto a più lingue e valute è diventato uno degli obiettivi del mio progetto (sezione 2.4). La seconda è che ogni cliente ha un'identità visiva propria, che vuole vedere rispettata anche negli strumenti che aggiunge al negozio: ne discuto nella sezione 2.2 e nella sezione 3.5.

1.3 Ecosistema tecnologico

Gli *storefront*$_G$, cioè le interfacce dei negozi Shopify rivolte ai clienti, possono seguire due modalità di sviluppo. La prima usa i temi della piattaforma: Shopify genera le pagine a partire da modelli scritti in Liquid$_G$, il suo linguaggio di *template*, combinandoli con i dati del negozio. La seconda adotta un'architettura *headless*$_G$, in cui l'interfaccia è un'applicazione separata che accede alle funzioni commerciali tramite API$_G$, interfacce di comunicazione tra programmi. Per questa modalità Shopify offre Hydrogen$_G$, un *framework* basato su React, e Oxygen, il servizio che ospita l'applicazione; Hydrogen non è però l'unico strumento possibile per costruire uno *storefront headless*.

Le Admin API permettono alle applicazioni di gestire i dati e le operazioni del negozio; le Storefront API espongono catalogo, carrello e altre funzioni dell'esperienza d'acquisto. Entrambe offrono interfacce GraphQL$_G$, un linguaggio di interrogazione con cui chi invia una richiesta specifica i campi da ottenere. I temi Liquid accedono invece ai dati anche tramite gli oggetti che Shopify rende disponibili durante la generazione delle pagine: non richiedono necessariamente chiamate esplicite a queste API. Per aggiungere funzionalità, Halue sviluppa applicazioni personalizzate in Node.js e TypeScript, integrabili con entrambe le modalità.

La tabella 1.2 distingue l'ambito di utilizzo delle tecnologie ricorrenti in Halue dal loro impiego nel progetto di *stage*.

Tabella 1.2: Le tecnologie ricorrenti nei progetti di Halue e il loro utilizzo nel progetto di *stage*.

Tecnologia	**Ambito di utilizzo**	**Utilizzo nel progetto di *stage***
Shopify e Shopify Plus	Gestione di negozi *e-commerce* B2C e B2B	Negozio Shopify di sviluppo
Temi Liquid	Generazione delle pagine del negozio	Inserimento di ChatCart nel tema
Admin API e Storefront API	Gestione del negozio e funzioni di acquisto tramite GraphQL	Accesso al catalogo con Storefront API
Node.js e TypeScript	Sviluppo di applicazioni personalizzate	Implementazione di ChatCart
Hydrogen e Oxygen	Sviluppo e pubblicazione di *storefront headless*	Non utilizzati

Tecnologia	Ambito di utilizzo	Utilizzo nel progetto di *stage*
Salesforce	Commercio digitale, CRM, assistenza e campagne commerciali	Non utilizzato
Storyblok e Sanity	Gestione dei contenuti separata dalla loro presentazione	Non utilizzati
Heroku, Google Cloud e AWS	Esecuzione di applicazioni e servizi in infrastrutture *cloud*	Non utilizzati; rilascio su Vercel

Per il mio progetto ho scelto i temi Liquid per la loro maggiore diffusione rispetto agli *storefront* costruiti con Hydrogen. Questa scelta permette di integrare ChatCart nei negozi esistenti senza sostituirne l'interfaccia.

Nei progetti di intelligenza artificiale cambia soprattutto l'architettura. Halue collega un LLM, di solito Claude, ai dati del negozio tramite MCP$_G$, un protocollo aperto che collega applicazioni basate su modelli linguistici a strumenti e dati esterni, e affianca al modello strumenti e istruzioni sviluppati internamente. Segue inoltre i nuovi protocolli per il commercio agentico, come UCP$_G$, pensati per permettere a un agente di completare un acquisto per conto dell'utente. Questa preferenza per strumenti propri l'ho vissuta direttamente: quando abbiamo definito il progetto, il *tutor* mi ha consigliato di non affidarmi al *server* MCP di Shopify, che considerava ancora troppo semplice, e di sviluppare una ricerca nel catalogo su misura. È la scelta da cui nasce il contratto tra agente e *widget* che descrivo nella sezione 3.3.2.

1.4 Processi di sviluppo e gestione dei progetti

Ho partecipato al coordinamento quotidiano del *team*, ma ho lavorato su un progetto interno, senza seguire l'intero ciclo di una commessa per un cliente.

1.4.1 Organizzazione del lavoro e gestione dei progetti

La cosa che mi ha colpito di più è il contrasto tra le dimensioni dell'azienda e il suo metodo. Halue ha dieci persone, ma adotta *framework* di lavoro molto strutturati, che di solito si associano ad aziende più grandi; allo stesso tempo mantiene la velocità di decisione di un'azienda piccola.

Il metodo di riferimento è Scrum$_G$, un *framework* per organizzare il lavoro in cicli brevi, adattato al lavoro di un'agenzia. Il lavoro procede per *sprint*$_G$, cicli brevi al termine dei quali il *team* presenta i risultati al cliente in una *sprint review* e ne discute internamente in una retrospettiva. Nella gestione operativa Halue distingue il *team lead*$_G$, che cura l'analisi funzionale, le stime per il cliente e il coordinamento tecnico, dal *project/service manager*$_G$, che pianifica le attività e la capacità del *team*. A questi si affiancano gli sviluppatori e chi progetta l'interfaccia. Nella mia lettura, l'adattamento sta soprattutto qui: non compaiono le figure canoniche di Scrum, il *Product Owner* e lo *Scrum Master*, e le loro responsabilità si distribuiscono tra il *team lead* e il *project/service manager*. È una scelta coerente con un'agenzia, in cui lo stesso gruppo di persone lavora per più clienti nello stesso periodo.

Per la stessa ragione le persone non sono assegnate a un progetto in modo stabile. Ogni mese un *demand planning* condiviso ripartisce il *team* sui clienti, e l'avanzamento di ciascun progetto si segue sulla *board* di Jira e con un rapporto sullo stato di avanzamento dei lavori (SAL)$_G$, che riporta con un semaforo lo stato del progetto, le scadenze, i rischi e i punti aperti. La figura 1.2 mette in fila questi momenti, dal più frequente al più raro.



Figura 1.2: I momenti di coordinamento del lavoro in Halue, ordinati per frequenza: la riunione quotidiana, le attività di ogni *sprint*, la pianificazione mensile delle persone sui clienti e i rapporti di avanzamento e le retrospettive di ogni progetto.

Il primo livello della figura è l'unico che ho vissuto ogni giorno. Ogni mattina tutto il *team* si collega su Google Meet per una riunione di allineamento: ciascuno dice su cosa ha lavorato il giorno prima e su cosa lavorerà quel giorno. La riunione dura di solito tra i quindici e i venti minuti, al massimo mezz'ora. Ho partecipato anch'io, con lo stesso formato degli altri.

Gli orari e il luogo di lavoro sono flessibili. Da quello che ho osservato, i colleghi lavorano in sede per circa il 70% del tempo e da remoto per il resto, ma ognuno gestisce la propria presenza in autonomia. Io sono stato in sede in media un giorno a settimana, come previsto dal piano di lavoro, e ho lavorato da remoto negli altri giorni.

La tabella 1.3 elenca gli strumenti di lavoro del *team* e distingue quelli che ho utilizzato da quelli che ho visto usare o che non ho impiegato direttamente.

Tabella 1.3: Gli strumenti di lavoro di Halue, con il loro uso e il modo in cui li conosco: usati durante lo *stage*, visti usare dai colleghi o soltanto riferiti dal *tutor*.

Strumento	**Uso in Halue**	**Come lo conosco**
Slack	Comunicazione interna e canali dedicati ai clienti	Usato, in un canale con il *tutor*
Google Meet	Riunioni quotidiane e *call*	Usato
Jira	Tracciamento delle unità di lavoro (*ticket*$_G$), *board* e stime	Visto, spiegato da un collega
Confluence	Documentazione e procedure	Visto
GitHub e Bitbucket	Versionamento del codice	GitHub usato, Bitbucket riferito
Figma	Progetto grafico e passaggio agli sviluppatori	Visto
Granola	Appunti automatici delle *call*	Visto
Postman	Prova delle API	Riferito

Con i clienti il *team* comunica su canali Slack dedicati a ciascun marchio, nelle *sprint review* con dimostrazioni del lavoro svolto e con rapporti periodici di avanzamento; i rapporti sulle attività e le variazioni contrattuali passano invece per *email*. La conoscenza interna sta su Confluence e su Google Drive. Una persona nuova parte dalla documentazione e dalle procedure e comincia con *ticket* piccoli su un progetto reale, con l'affiancamento di un collega. Il mio inserimento è stato diverso, perché lavoravo su un progetto mio: ho imparato i processi dell'azienda facendo domande e ascoltando le riunioni, più che lavorandoci dentro.

1.4.2 Dal *ticket* al rilascio

Le richieste arrivano da *call*, *email* o *workshop* e diventano *ticket* su Jira, di quattro tipi: *Epic* per gli insiemi di funzionalità, *Story* per le singole funzionalità, *Change Request* per le modifiche chieste a progetto avviato e *Bug* per i difetti. Ogni *ticket* passa per due fasi di approfondimento: il *refinement* funzionale, che ne valuta la completezza e la fattibilità, e il *refinement* tecnico, svolto dagli sviluppatori. I *bug* di norma saltano il secondo passaggio.

Le stime sono due, perché servono a due destinatari. Per il *team* si usano gli *story point*$_{\text{G}}$, una misura della complessità relativa espressa con i numeri di Fibonacci da 1 a 13: servono a pianificare lo *sprint*, e un *ticket* che supera 13 va spezzato in *ticket* più piccoli. Per il cliente si stimano invece le ore, con un metodo interno che pesa quattro dimensioni: la complessità, la chiarezza della richiesta, il *team* coinvolto e la confidenza nella stima.

Il progetto grafico arriva da Figma, collegato al *ticket*, sempre in versione per dispositivi mobili e per *desktop*. Il passaggio agli sviluppatori avviene in Figma stesso, con le specifiche e con un confronto diretto con il *designer* sui punti meno chiari, e il *designer* rivede l'implementazione prima del rilascio. Ogni marchio ha il proprio *design system*$_{\text{G}}$, cioè l'insieme di componenti e stili che rende coerenti le sue pagine: è il motivo per cui, nel mio progetto, adattare l'aspetto del *widget* al negozio si è rivelato così difficile (sezione 3.5).

Nello sviluppo ogni *ticket* corrisponde a una *pull request*$_{\text{G}}$, una proposta di integrazione delle modifiche al codice, e i nomi dei *branch* e i messaggi di *commit* riportano la chiave del *ticket*, così che ogni modifica al codice si possa ricondurre alla richiesta da cui nasce. ESLint e Prettier controllano lo stile del codice, e le versioni seguono il versionamento semantico (SemVer)$_{\text{G}}$, che distingue modifiche incompatibili, nuove funzionalità e correzioni, con *tag* e *release* su GitHub. La verifica combina *test* automatici, di unità e di integrazione, e *test* manuali; non c'è una copertura minima prefissata. Il controllo di qualità lo svolgono gli sviluppatori durante lo sviluppo, e poi il cliente nel collaudo.

Il rilascio attraversa tre ambienti: sviluppo, un ambiente di collaudo (*staging*$_{\text{G}}$ o *preview*) e produzione. Il *deploy* è automatizzato e segue una procedura scritta, ma il passaggio in produzione lo esegue sempre una persona del *team*. Dopo la consegna, i difetti e le richieste di modifica passano per i contratti di manutenzione: le segnalazioni arrivano su Jira, sul canale Slack del cliente o per *email*, e le priorità si concordano con il cliente caso per caso, senza livelli di servizio formali.

Le decisioni di architettura nascono nel *refinement* funzionale e si consolidano in quello tecnico. Quelle che riguardano un singolo *ticket* restano scritte nel *ticket*, dove chi sviluppa le trova; quelle che valgono per tutto il progetto diventano ADR_G, documenti brevi che registrano il contesto, la scelta e la sua motivazione. Il ciclo si chiude con le retrospettive, a fine *sprint* e alla chiusura di un progetto o di una *release* importante, per portare le correzioni nei progetti successivi.

La figura 1.3 riassume questo percorso, dalla richiesta del cliente al rilascio in produzione e alla manutenzione, mostrando i passaggi di analisi, sviluppo e verifica.

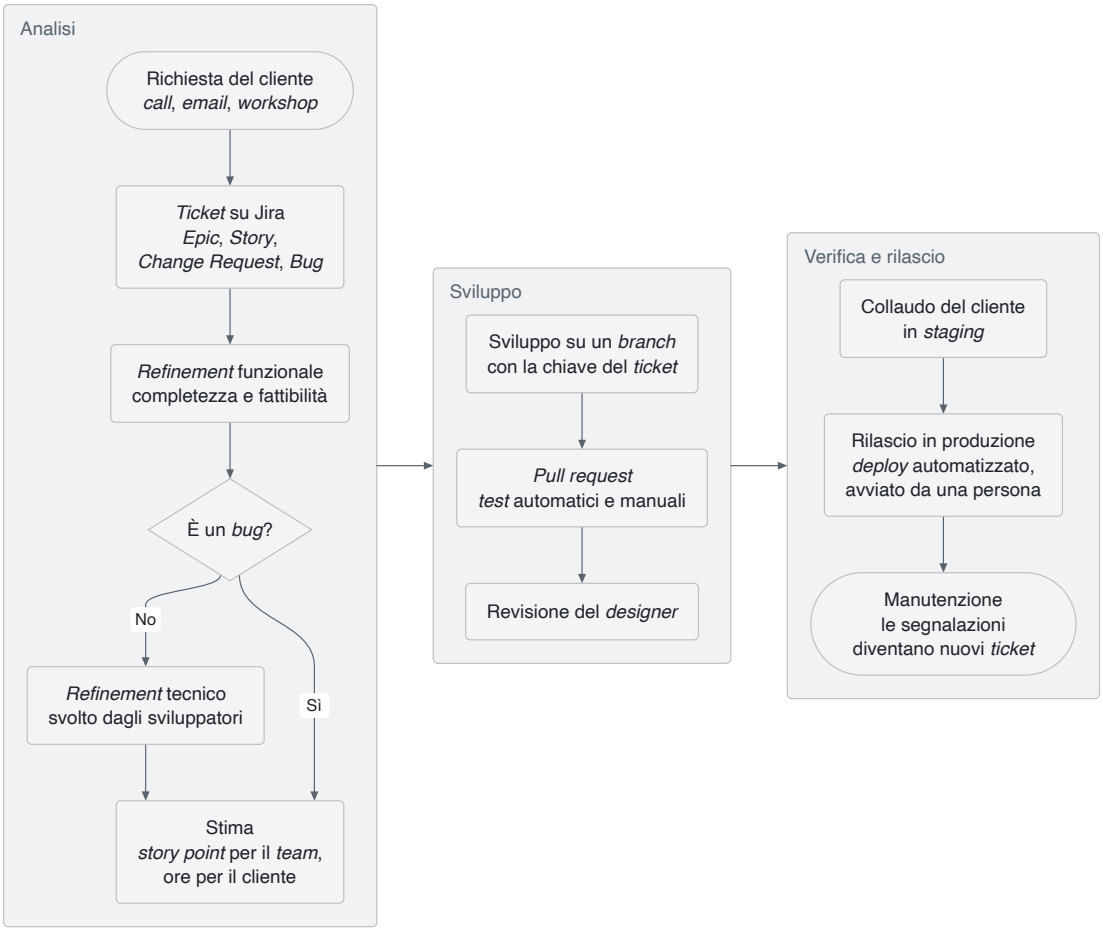


Figura 1.3: Il ciclo di vita di un *ticket* in Halue: dalla richiesta del cliente al *ticket* su Jira, attraverso i due *refinement*, la stima, lo sviluppo con *pull request* e *test*, il collaudo in *staging*, fino al rilascio in produzione e alla manutenzione.

Da questo quadro ricavo una lettura del metodo di Halue: la struttura è rigida dove si ripaga sempre, cioè nella tracciabilità dalla richiesta al codice e nelle stime, e flessibile dove i clienti sono diversi tra loro, come la copertura dei *test* o i livelli di servizio, che si decidono progetto per progetto. Ho trovato la stessa impostazione nel

mio *stage*, per esempio nella scelta di concentrare i *test* automatici dove servivano davvero (sezione 3.6).

1.5 Innovazione e intelligenza artificiale

L'azienda dichiara sul proprio sito di fare uno *scouting* tecnologico continuo, di ridurre il debito tecnico allungando la vita delle piattaforme e di dedicare oltre il 10% del tempo a progetti di ricerca e sviluppo, tra cui quelli di intelligenza artificiale. Non ho verificato la percentuale, ma ho visto quanto tempo i colleghi dedicano a studiare: lo considerano un modo per crescere come professionisti, non soltanto come dipendenti dell'azienda. Halue organizza inoltre una formazione interna sui metodi agili, sulle piattaforme e sulla tecnologia, e ogni anno qualche evento formativo per i clienti.

L'intelligenza artificiale è lo strumento in cui questa propensione si vede di più. Il *team* usa ampiamente strumenti di sviluppo basati sull'IA e segue da vicino tutto ciò che esce nel settore: ho visto *workflow* di *code review* automatica, la lettura dei *changelog* delle tecnologie in uso, *routine* ricorrenti con Claude Cowork e Claude Code, e ho sentito parlare di prove con altri agenti, come Codex di OpenAI. L'uso quotidiano dell'IA comprende anche la scrittura della documentazione e dei *ticket*, la gestione dei progetti, per i rapporti e le stime, e il progetto grafico. Per i processi ricorrenti l'azienda ha scritto *skill*$_{\text{G}}$ interne, istruzioni riusabili che codificano un processo per un assistente basato su IA.

La regola di fondo è che l'IA propone e la persona decide. L'azienda riscontra una riduzione dei tempi, ma non dispone ancora di misure precise su questo beneficio e sull'impatto sulla qualità. Ho percepito anche un'opinione condivisa nel *team*, che la mia esperienza conferma: nelle mani di un programmatore esperto l'IA è un buon programmatore *junior*, ma non sostituisce un *senior*, e chi sa cosa sta facendo ottiene risultati migliori di quelli dell'IA da sola. Torno su questo punto nella sezione 4.2.

Nei progetti di IA per i clienti Halue adotta alcune regole che ho ritrovato nel mio progetto. I dati vengono sempre dal catalogo reale, tramite strumenti e ricerca, e mai dalla memoria del modello; le azioni con effetti esterni restano sotto controllo umano; la qualità delle risposte si valuta con *test* manuali su casi d'uso reali; il modello si sceglie in base al compito, e i consumi di *token* si tengono sotto controllo.

Questa propensione all'innovazione passa anche dagli *stage*, che Halue usa per esplorare temi che i progetti per i clienti non lasciano il tempo di approfondire.

Approfondisco questo ruolo nella sezione 2.1.

Capitolo 2

Oltre la conversazione testuale

Discuto perché Halue ha proposto questo *stage* e perché io l'ho scelto: il problema che affronta, il suo posto nella strategia dell'azienda, gli obiettivi e i vincoli del progetto.

2.1 Il ruolo degli *stage* nella strategia aziendale

Descrivo il rapporto di Halue con gli *stage*: il numero di stagisti che ospita ogni anno, la loro provenienza e il duplice scopo che l'azienda attribuisce a queste esperienze, cioè esplorare temi che i progetti per i clienti non lasciano il tempo di approfondire e conoscere persone con cui collaborare in futuro. Illustro come nasce un progetto di *stage* e quale destino può avere il lavoro al suo termine, e collego queste scelte alla propensione all'innovazione discussa nella sezione 1.5.

2.2 I limiti degli assistenti conversazionali testuali

Introduco il problema che lo *stage* affronta. I *chatbot* integrati nei negozi *online* rispondono quasi esclusivamente in forma testuale, una modalità poco efficace quando il cliente deve esplorare un catalogo, confrontare prodotti o scegliere una variante. Una figura mette a confronto la risposta solo testuale di un *chatbot* con quella di ChatCart alla stessa domanda. Esamino le alternative disponibili: i *chatbot* generici, Shopify Sidekick, rivolto al commerciante più che al cliente, e i protocolli con cui Shopify apre il negozio agli agenti, come il suo *server* MCP e UCP. Discuto infine perché le soluzioni preconfezionate si adattano con difficoltà all'identità del marchio e alle specificità del suo catalogo.

2.3 Il commercio agentico nella visione di Halue

Colloco lo *stage* nella strategia dell'azienda, che intende costruire competenze sul commercio agentico, in cui un agente di intelligenza artificiale accompagna il cliente fino all'acquisto. Descrivo ciò che viene prima dello *stage*: i progetti di Halue su Shopify Plus e gli agenti collegati ai dati del negozio con strumenti propri. Descrivo poi ciò che viene dopo: la presentazione finale al *team*, la scelta tra congelare il lavoro e riusarne alcune parti, l'obiettivo di proporre questo tipo di soluzione ai clienti come servizio, con tempi e forma ancora da decidere, e la possibilità che io prosegua in azienda.

2.4 Obiettivi e vincoli del progetto

Ricostruisco la genesi del progetto: Halue mi ha proposto due temi, l'interfaccia generativa e la gestione del contesto, e io ho scelto il primo, proponendo un agente in grado di selezionare e popolare componenti dell'interfaccia. Presento l'architettura su due livelli prevista dal piano di lavoro, gli obiettivi obbligatori, desiderabili e facoltativi, riassunti in una tabella con i loro identificativi, da OB1 a FA2, la ripartizione delle ore e i vincoli: otto settimane di durata, il lavoro in parte da remoto, un unico negozio di sviluppo e la natura di prototipo di ricerca, non di prodotto.

2.5 Motivazioni e obiettivi personali

Espongo le ragioni della mia scelta. Da marzo 2026 lavoro in proprio come sviluppatore Shopify, ma la mia esperienza riguardava soltanto i temi Liquid e non lo sviluppo di applicazioni: per questo ho scelto Halue non appena l'ho individuata tra le proposte. L'idea del progetto nasce da un'esigenza concreta di un mio cliente, che cercava un *chatbot* collegato al negozio e capace di mostrare prodotti, non soltanto testo. Da queste ragioni ricavo i miei obiettivi personali, ciascuno con un identificativo che riprendo nel capitolo 4: ampliare le mie competenze sull'ecosistema Shopify, acquisire esperienza nella progettazione di sistemi basati su agenti e ricavare dal progetto un valore concreto per la mia attività professionale.

Capitolo 3

Progetto e sviluppo di ChatCart

Racconto lo *stage* come un piccolo progetto a sé stante: il metodo di lavoro, l'analisi, la progettazione, la realizzazione, le sperimentazioni, la verifica e i risultati.

3.1 Metodo di lavoro

3.1.1 Pianificazione e interazione con il *tutor* aziendale

Confronto le modalità di interazione previste dal piano di lavoro con quelle effettive: le riunioni quotidiane, i resoconti di fine giornata su Slack, la riunione settimanale, i confronti su richiesta e le giornate in sede. Discuto le difficoltà della fase iniziale e il modo in cui le ho superate. Descrivo infine i resoconti settimanali al relatore e la svolta della terza settimana: con gli obiettivi obbligatori già soddisfatti, io e il *tutor* abbiamo deciso di dedicare il resto dello *stage* alla sperimentazione. Un diagramma temporale mette a confronto la pianificazione originaria del piano di lavoro con il modo in cui ho usato davvero il tempo, attività per attività.

3.1.2 Organizzazione del lavoro e tracciamento delle attività

Illustro come ho organizzato il mio lavoro, svolto in gran parte da remoto: come individuavo le attività e ne stabilivo le priorità, come ne tenevo traccia e come preparavo le modifiche più estese prima di realizzarle.

3.1.3 Strumenti e assistenti di programmazione

Presento gli strumenti che ho impiegato per lo sviluppo, la verifica e il rilascio, tra cui gli assistenti di programmazione basati su modelli linguistici. Di questi descrivo l'uso che ne ho fatto: la sperimentazione e la validazione rapida delle idee, fino a prototipi funzionanti nell'arco di una giornata, a cui seguiva l'implementazione definitiva scritta a mano; la stesura dei documenti di progettazione, per individuare in anticipo i casi limite e i *test* necessari; le attività di supporto, come la generazione dei dati di prova. Ciò che ho imparato da questo uso lo discuto nella sezione 4.2.

3.2 Studio preliminare e analisi

3.2.1 L'ecosistema delle applicazioni Shopify

Descrivo la fase iniziale dello *stage*, dedicata allo studio del funzionamento di un'applicazione Shopify (autenticazione, *app proxy*, estensioni del tema, differenze tra Storefront API e Admin API) e degli strumenti per la costruzione di agenti. Distinguo le conoscenze che avevo già acquisito lavorando sui temi Liquid da quelle che ho dovuto costruire per la prima volta.

3.2.2 Un'analisi dei requisiti iterativa

Motivo la scelta, condivisa con il *tutor*, di non svolgere un'analisi dei requisiti completa all'inizio del progetto: il lavoro aveva carattere esplorativo, nessun cliente ne dettava i requisiti, e una specifica fissata a priori sarebbe diventata presto obsoleta. Descrivo il procedimento che ho seguito: a partire dagli obiettivi del piano ho abbozzato i casi d'uso principali e ne ho ricavato i requisiti, che ho rivisto a ogni iterazione insieme al prodotto. Un diagramma mostra i casi d'uso nella loro forma finale e una tabella riporta i requisiti a cui sono arrivato, raggruppati per area, ciascuno con un identificativo e l'obiettivo del piano da cui deriva; ne misuro la copertura nella sezione 3.7.2.

3.3 Architettura del sistema

3.3.1 Visione d'insieme

Illustro l'architettura di ChatCart con un diagramma: il *widget* nel negozio (*app layer*), il *server* che ospita l'agente (*agent layer*), la *dashboard* del commerciante e i

servizi esterni. Motivo la scelta di uno stile funzionale anziché orientato agli oggetti, la decisione, su indicazione del *tutor*, di non introdurre astrazioni prima che fossero necessarie e i *design pattern* che ho adottato quando lo sono diventate.

3.3.2 Il contratto tra agente e *widget*

Approfondisco la scelta su cui si regge l'interfaccia generativa: un unico elenco di strumenti, condiviso tra agente e *widget*, che distingue gli strumenti che leggono i dati del negozio da quelli che mostrano un componente e chiudono la risposta. Spiego come da questo elenco derivo i tipi condivisi tra le due parti, così che il *widget* sappia sempre disegnare ciò che l'agente sceglie.

3.4 Realizzazione

3.4.1 L'agente conversazionale

Descrivo il processo con cui l'agente costruisce una risposta: un ciclo in cui il modello invoca gli strumenti per consultare il catalogo, finché non seleziona un componente da mostrare. Discuto come ho ripartito il controllo del comportamento tra le istruzioni al modello e le regole fisse nel codice, come ho generato i tre messaggi suggeriti e come ho scelto il modello e il livello di ragionamento, confrontandone velocità e accuratezza. Un diagramma di sequenza segue il percorso di un messaggio dall'utente alla risposta. Questa parte risponde agli obiettivi OB2 e OB3.

3.4.2 Il *widget* e l'integrazione con il tema del negozio

Presento il *widget*: i componenti (schede prodotto con selezione della variante, schede con le risposte tratte dalle pagine del negozio, collegamenti alle collezioni), il supporto a più lingue e valute e, in particolare, la sincronizzazione del carrello con il tema del negozio. Analizzo perché ogni tema Shopify gestisce il carrello in modo diverso e come ho affrontato il problema con un *adapter* per ciascun tema e una soluzione di ripiego. Questa parte risponde agli obiettivi OB1, OB4 e DE1.

3.4.3 Metriche di utilizzo e analisi delle conversazioni

Descrivo gli eventi che il *widget* registra, il *funnel* che la *dashboard* presenta al commerciante e l'analisi delle conversazioni con Jev, un modello decisionale di TypeSafe AI uscito durante lo *stage*: a differenza di un modello linguistico generativo, non produce testo ma restituisce decisioni tipizzate con la loro probabilità. Da qui apro

il confronto tra i due tipi di modello nei compiti decisionali. Questa parte risponde all'obiettivo OB5.

3.5 Sperimentazioni e sviluppi futuri

Dedico questa sezione alle funzionalità che ho sperimentato senza portarle nel prodotto finale, perché documentano l'orientamento alla ricerca che io e il *tutor* abbiamo dato al progetto dalla terza settimana: i messaggi di invito che comparivano prima dell'apertura della chat, i tre tentativi di adattare l'aspetto del *widget* al tema del negozio, l'impiego di Jev nella scelta dei componenti, il pannello a tutto schermo su *desktop*. Per ciascuna espongo l'ipotesi di partenza, l'esito che ho osservato e le ragioni per cui io e il *tutor* abbiamo deciso di non inserirla nel prodotto consegnato. Da queste prove ricavo infine le strade lungo cui ChatCart potrebbe evolvere in futuro.

3.6 Verifica e validazione

Illustro la mia strategia di verifica: *test* automatici concentrati sui punti in cui ChatCart interagisce con sistemi esterni che non controllo, eseguiti su una base di dati reale e senza simulare i moduli interni, e le ragioni di questa scelta. Discuto i limiti dei *test* tradizionali applicati a un agente che non risponde mai due volte nello stesso modo. Concludo con la validazione svolta insieme al *tutor* e con il rilascio dell'applicazione *online*.

3.7 Risultati

3.7.1 Il prodotto dal punto di vista dell'utente

Presento ChatCart dal punto di vista di chi lo utilizza, attraverso una sequenza di schermate: un cliente che cerca un prodotto, lo confronta con altri, lo aggiunge al carrello e raggiunge il pagamento; poi il commerciante che consulta il *funnel* e le conversazioni nella *dashboard*. Dichiaro inoltre i limiti noti del prototipo.

3.7.2 Risultati quantitativi

Riporto in tabelle i dati quantitativi del progetto: la copertura dei requisiti definiti nella sezione 3.2.2, la copertura dei *test* con le scelte che la motivano, le righe di

codice per ciascuna parte dell'applicazione e i documenti che ho consegnato al *team*.

Capitolo 4

Valutazione retrospettiva

Ripercorro lo *stage* a posteriori: in che misura ho raggiunto gli obiettivi, che cosa ho imparato e quanto la formazione universitaria mi abbia preparato al lavoro che ho svolto.

4.1 Soddisfacimento degli obiettivi

Valuto, sulla base di dati di fatto, il grado di soddisfacimento degli obiettivi presentati nella sezione 2.4 e nella sezione 2.5: una tabella riporta ciascun obiettivo del piano, da OB1 a FA2, e ciascuno dei miei obiettivi personali, con il relativo stato e l'evidenza che lo documenta. Valuto allo stesso modo i prodotti attesi: quelli elencati nel piano di lavoro e la presentazione formale al *team* che il *tutor* mi ha chiesto a fine *stage*. Discuto gli scostamenti, come gli obiettivi che io e il *tutor* abbiamo scartato e i limiti delle metriche raccolte.

4.2 Maturazione professionale

Descrivo la mia maturazione professionale in termini di conoscenze, come l'ecosistema delle applicazioni Shopify e la costruzione di agenti, e di abilità, come il lavoro da remoto, la comunicazione con il *tutor*, la gestione del tempo e la capacità di stabilire cosa escludere dal progetto. Rifletto inoltre su ciò che ho appreso dall'uso dell'intelligenza artificiale nella programmazione, il tema su cui si è concentrato l'incontro conclusivo con il fondatore.

4.3 Formazione universitaria e lavoro

Confronto le competenze che lo *stage* mi ha richiesto con quelle che ho acquisito nel corso di studi. Individuo quelle che si sono rivelate utili e le lacune che sono emerse, come la progettazione di agenti e la verifica di sistemi dal comportamento non deterministico, sia rispetto allo *stage* sia rispetto all'idea che ho del mondo del lavoro.

Acronimi e abbreviazioni

Elenco in ordine alfabetico gli acronimi e le abbreviazioni introdotti nel capitolo 1. Quando presente, il collegamento rimanda all'approfondimento nel glossario.

ADR: *Architecture Decision Record*.

API: *Application Programming Interface*.

AWS: Amazon Web Services.

B2B: *Business-to-Business*.

B2C: *Business-to-Consumer*.

CMS: *Content Management System*.

CRM: *Customer Relationship Management*.

ERP: *Enterprise Resource Planning*.

IA: intelligenza artificiale.

iPaaS: *Integration Platform as a Service*.

LLM: *Large Language Model*.

MCP: *Model Context Protocol*.

SAL: stato di avanzamento dei lavori.

SemVer: *Semantic Versioning*.

UCP: *Universal Commerce Protocol*.

UI: *User Interface*.

UX: *User Experience*.

Glossario

Raccolgo in ordine alfabetico i termini tecnici introdotti nel testo. Alla prima occorrenza nel corpo della relazione, il termine e il pedice G rimandano alla voce corrispondente.

ADR Documento che registra una decisione di architettura: il contesto in cui nasce, le alternative considerate, la scelta e le sue conseguenze. Conserva le motivazioni della scelta anche quando cambia chi lavora al progetto.

API Interfaccia con cui un programma rende disponibili dati o operazioni ad altri programmi, secondo un contratto definito. In Shopify, la Admin API permette di amministrare il negozio; la Storefront API espone le funzioni rivolte all'esperienza d'acquisto.

B2B Modello commerciale in cui un'azienda vende prodotti o servizi ad altre aziende. Nel commercio elettronico può richiedere cataloghi, listini e condizioni di acquisto diversi per ciascun cliente.

B2C Modello commerciale in cui un'azienda vende prodotti o servizi al consumatore finale. È il contesto dell'interfaccia conversazionale di ChatCart, rivolta a chi acquista nel negozio.

CRM Insieme di pratiche e sistemi per gestire le relazioni di un'azienda con i propri clienti. Un sistema CRM raccoglie, per esempio, contatti, interazioni commerciali e richieste di assistenza.

Design system Insieme coerente di principi, regole visive e componenti riusabili per progettare un'interfaccia. Comprende elementi come colori, tipografia, spaziature e comportamenti dei componenti, così da mantenere riconoscibile l'identità del prodotto o del marchio.

E-commerce Commercio di prodotti o servizi attraverso canali digitali. Un negozio elettronico comprende il catalogo, la selezione dei prodotti, il carrello e il pagamento, oltre ai collegamenti con i sistemi che gestiscono ordini e logistica.

ERP Sistema gestionale che integra dati e processi di diverse aree aziendali, come amministrazione, acquisti, magazzino e vendite. Il collegamento con il negozio permette, per esempio, di sincronizzare ordini e disponibilità dei prodotti.

Founder & Senior Consultant Ruolo che unisce la fondazione dell'azienda all'attività di consulenza esperta. In Halue lo ricopre Valentino Baraldo.

GraphQL Linguaggio di interrogazione per API e insieme di regole per eseguire le richieste su uno schema di dati tipizzato. Il programma che invia la richiesta specifica i campi di cui ha bisogno; Shopify lo adotta per la Admin API e la Storefront API.

Headless Architettura che separa la gestione dei dati e delle funzioni applicative dall'interfaccia che li presenta. In un negozio di questo tipo l'interfaccia è un'applicazione autonoma che comunica con la piattaforma commerciale tramite API; in un CMS la stessa separazione riguarda i contenuti e la loro presentazione.

Hydrogen *Framework* di Shopify basato su React per costruire interfacce di negozi *headless*. Offre strumenti e componenti per accedere ai dati del negozio tramite la Storefront API; Oxygen è il servizio di Shopify che può ospitare queste applicazioni.

iPaaS Piattaforma erogata come servizio per collegare applicazioni e sistemi diversi. Offre connettori e strumenti per trasformare e trasferire i dati, riducendo la necessità di costruire un'integrazione distinta per ogni coppia di sistemi.

Junior Developer Sviluppatore all'inizio del proprio percorso professionale, che realizza e verifica funzionalità con il supporto di colleghi più esperti.

Liquid Linguaggio di *template* usato nei temi Shopify. Permette di combinare la struttura delle pagine con i dati del negozio, attraverso variabili, condizioni e cicli; la piattaforma elabora i modelli e genera le pagine da inviare al navigatore.

LLM Modello linguistico di grandi dimensioni, addestrato su ampie raccolte di dati per elaborare e generare testo. In un agente può selezionare strumenti esterni

di cui l'applicazione esegue le operazioni; il collegamento a questi strumenti gli permette di usare dati aggiornati che non fanno parte del suo addestramento.

MCP Protocollo aperto che standardizza la comunicazione tra applicazioni basate su modelli linguistici e servizi esterni. Un *server* MCP può esporre strumenti, risorse e istruzioni; è l'applicazione che ospita il modello a gestire il collegamento e l'esecuzione delle richieste.

Project Management Assistant Figura che supporta la gestione dei progetti, contribuendo all'organizzazione delle attività, all'aggiornamento della documentazione e al monitoraggio delle scadenze.

Project/service manager Responsabilità operativa di gestione di un progetto o di un servizio continuativo. Nel metodo di Halue cura la pianificazione delle attività e la capacità disponibile del gruppo di lavoro.

Pull request Proposta di integrare un insieme di modifiche da un *branch* a un altro in un sistema di versionamento. Raccoglie le differenze nel codice, la discussione di revisione e gli esiti dei controlli automatici prima dell'integrazione.

SAL Stato di avanzamento dei lavori. Nel contesto descritto indica un rapporto periodico che riassume il progresso del progetto, le scadenze, i rischi e i punti aperti, per condividere la situazione con le persone coinvolte.

Scrum *Framework* per affrontare problemi complessi attraverso cicli brevi di lavoro, chiamati *sprint*, e un confronto regolare sui risultati. Prevede responsabilità, eventi e artefatti definiti; nella sezione 1.4.1 distinguo questo riferimento dall'adattamento che Halue ne fa per il lavoro su commessa.

SemVer Convenzione di versionamento semantico che esprime una versione nella forma `MAJOR.MINOR.PATCH`. Rispetto all'API pubblica dichiarata, il primo numero cambia per modifiche incompatibili, il secondo per nuove funzionalità compatibili e il terzo per correzioni compatibili.

Senior Project Manager Figura con esperienza nella gestione dei progetti, responsabile del coordinamento di pianificazione, risorse e avanzamento delle attività.

Senior Software Engineer Professionista dello sviluppo del *software* con esperienza consolidata, che affronta problemi progettuali complessi e contribuisce alle scelte tecniche e alla revisione del codice.

Skill Insieme riusabile di istruzioni e, quando necessari, risorse o strumenti, che guida un assistente basato su IA in un'attività specifica. Serve a rendere esplicito un procedimento, ma non garantisce da solo la correttezza dell'esecuzione.

Società Benefit Società che affianca allo scopo di lucro una o più finalità di beneficio comune indicate nello statuto. La disciplina italiana richiede una gestione che bilanci questi obiettivi e una relazione annuale sull'impatto prodotto; la qualifica non coincide con una certificazione privata di sostenibilità.

Software Engineer Professionista che applica metodi di ingegneria del *software* alla progettazione, realizzazione, verifica e manutenzione di sistemi *software*.

Solution & Technical Architect Figura che traduce le esigenze applicative in una soluzione tecnica, definendo l'architettura del sistema, le tecnologie e le integrazioni tra componenti.

Sprint Ciclo di lavoro di durata fissa, non superiore a un mese nel riferimento Scrum, durante il quale il gruppo persegue un obiettivo concordato. La revisione dei risultati e la retrospettiva permettono di adattare il prodotto e il modo di lavorare al ciclo successivo.

Staging Ambiente separato dalla produzione in cui collaudare una versione dell'applicazione prima di renderla disponibile agli utenti. Riproduce le condizioni di esercizio per quanto necessario alla verifica, senza coincidere con il servizio effettivamente usato dai clienti.

Storefront Interfaccia di un negozio digitale rivolta al cliente, attraverso cui consultare il catalogo, selezionare prodotti e avviare l'acquisto. In Shopify può basarsi sui temi della piattaforma oppure su un'applicazione separata con architettura *headless*.

Story point Unità di stima relativa che un gruppo usa per confrontare le attività in termini di complessità, impegno e incertezza. Non corrisponde a un numero fisso di ore e acquista significato rispetto alle stime dello stesso gruppo; la scala di Fibonacci è una possibile convenzione, non un obbligo di Scrum.

Team lead Responsabilità di guida tecnica di un gruppo di lavoro. In Halue comprende l'analisi funzionale, le stime per il cliente e il coordinamento tecnico degli sviluppatori.

Ticket Unità di lavoro registrata in un sistema di tracciamento, con un identificativo, una descrizione e uno stato. Può rappresentare una funzionalità,

una modifica o un difetto, e raccogliere stime, discussioni e collegamenti alle modifiche del codice.

UCP Protocollo aperto per rendere interoperabili i servizi commerciali e gli agenti che operano per conto dell'utente. Definisce modalità condivise per scoprire le capacità di un negozio e interagire con le fasi dell'acquisto; riguarda il dominio commerciale, mentre MCP offre un meccanismo generale per collegare applicazioni e strumenti.

UX/UI Digital Consultant Consulente che progetta l'esperienza utente e l'interfaccia di un prodotto digitale, considerando percorsi d'uso, struttura delle informazioni e aspetto visivo.

Riferimenti

- Halue S.r.l., sito istituzionale, https://halue.com/it, pagine «Chi siamo», «Approccio» e «Progetti», consultate il 4 novembre 2026.